

Asynchronous JavaScript 2

Brainster Web Development Academy



Table of contents

- 01 POST (and other) with the Fetch API
- 02 async/await
- 03 Promise.all
- 04 Promise.race
- 05 HTTP status codes



POST (and others) with the Fetch API

- Fetch also has an alternative form of sending request - using a configuration objects instead of using a URL directly.
- This method signature allows us for a more flexible approach - namely, changing the verb of the request (GET, POST, PUT ...), setting headers, changing data types etc.

```
fetch('https://jsonplaceholder.typicode.com/posts', {  
  method: 'POST' // can be GET, POST, PUT, PATCH, DELETE...  
})  
  .then((response) => response.json())  
  .then((data) => { console.log(data) // use the response data here})
```



Example 1

1. Send a POST request to typicode:
 - a. Use <https://jsonplaceholder.typicode.com/posts> as the URL
 - b. The data you send should be an object that contains: id, title, body and userId (id and userId are numbers, title and body are strings)
 - c. First, log the response from the POST request you send
 - d. Then, show the new post on screen, using the new title and body from the response



Example 2

1. Create a blog.
 - a. Load one blogpost from this url <https://jsonplaceholder.typicode.com/posts/id>
2. Under the blogpost, there should be a button that says 'Load more'
3. Each time you press the button, one more post is loaded (added).
4. The page and the posts should be styled.



Example 3

1. Create a paginated blog.
 - a. Load one blogpost from this url https://jsonplaceholder.typicode.com/posts?_limit=5&_page=1
2. It should list 5 posts per page.
3. It should have 'Previous' and 'Next' buttons for loading more posts.
4. The 'Previous' button should be disabled on the first page



async/await

- ES7 introduced `async` and `await` syntax.
- It makes the asynchronous syntax look prettier and easier to understand, without the `.then` and `.catch`.
- Async functions.
- The word “`async`” before a function means one simple thing: a function always returns a promise. Other values are wrapped in a resolved promise automatically.

5

```
async function f() {  
  return 1;  
}
```

is equal to

```
async function f() {  
  return Promise.resolve(1);  
}
```

```
f().then(alert); // 1
```

```
f().then(alert); // 1
```

- So, `async` ensures that the function returns a promise, and wraps non-promises in it.
- There's another keyword, **`await`**, that works only inside `async` functions, and it's pretty cool.



async/await

- The keyword `await` makes JavaScript wait until that promise settles and returns its result.
- **`await`** works only inside **`async`** functions

```
5  async function f() {  
    const promise = new Promise((resolve, reject) => {  
      setTimeout(() => resolve('done!'), 1000);  
    });  
  
    const result = await promise; // wait until the promise resolves (*)  
  
    alert(result);  
  }
```

- The function execution “pauses” at the line (*) and resumes when the promise settles, with result becoming its result. So the code above shows “done!” in one second.
- It’s just a more elegant syntax of getting the promise result than `promise.then`, easier to read and write.



async/await

- We can't use await in regular functions
- If we try to use await in non-async function, there would be a syntax error:

```
function f() {  
  const promise = Promise.resolve(1);  
  const result = await promise; // Syntax error  
}
```

5

- await won't work in the top-level code - we can't use await in top-level code.



Example 4

1. Re-write example 1 using `async/await`. (just the fetching part)
 2. Send a POST request to `typicode`:
 - a. Use <https://jsonplaceholder.typicode.com/posts> as the URL
 - b. The data you send should be an object that contains: `id`, `title`, `body` and `userId` (`id` and `userId` are numbers, `title` and `body` are strings)
- 3
- c. First, log the response from the POST request you send
 - d. Then, show the new post on screen, using the new title and body from the response



Example 5

3

1. Re-write example 2 using async/await. (just the fetching part)
2. Create a blog.
 - a. Load one blogpost from this url <https://jsonplaceholder.typicode.com/posts/id>
3. Under the blogpost, there should be a button that says 'Load more'
4. Each time you press the button, one more post is loaded (added).
5. The page and the posts should be styled.



Example 6

1. Re-write example 3 using async/await. (just the fetching part)
2. Create a paginated blog.
 - a. Load one blogpost from this url https://jsonplaceholder.typicode.com/posts?_limit=5&_page=1
3. It should list 5 posts per page.
4. It should have 'Previous' and 'Next' buttons for loading more posts.
5. The 'Previous' button should be disabled on the first page



Promise.all

- Let's say we want many promises to execute in parallel and wait until all of them are ready.
- For instance, download several URLs in parallel and process the content once they are all done.
- Promise.all takes an **array of promises** (it technically can be any iterable, but is usually an array) and returns a **single** new promise.
- The new promise resolves when all listed promises are settled, and the array of their results becomes its result.

6

```
Promise.all([
  new Promise(resolve => setTimeout(() => resolve(1), 3000)), // 1
  new Promise(resolve => setTimeout(() => resolve(2), 2000)), // 2
  new Promise(resolve => setTimeout(() => resolve(3), 1000)), // 3
]).then(alert); // 1, 2, 3 when promises are ready
```

- Result:
- Note that the order of the resulting array members is the same as in its source promises. Even though the first promise takes the longest time to resolve, it's still first in the array of results.



Example 7

1. Fetch all users from this api endpoint <https://jsonplaceholder.typicode.com/users>
2. Fetch all posts written by users from this api endpoint <https://jsonplaceholder.typicode.com/posts>
3. After we are sure that all data is loaded(fetching is done), display all posts grouped by user.



Promise.race

- Let's say we want many promises to execute in parallel and wait until the first one of them is ready.
- For instance, try to fetch something from the BE, but give up if 5 seconds pass.
- Promise.race takes an **array of promises** (it technically can be any iterable, but is usually an array) and returns a **single** new promise (similar to Promise.all).
- The new promise resolves when the first one from the listed promises is settled. The promise is then fulfilled and the result is the one from the 'winning' promise.

6

```
Promise.race([
  new Promise(resolve => setTimeout(() => resolve(1), 3000)), // 1
  new Promise(resolve => setTimeout(() => resolve(2), 2000)), // 2
  new Promise(resolve => setTimeout(() => resolve(3), 1000)), // 3
]).then(alert); // 3, because that promise will be the first one to resolve
```



Example 8

1. Fetch all users from this api endpoint <https://jsonplaceholder.typicode.com/users>
2. If the call takes longer than 2 seconds, show an alert that the call to BE failed.

hint: Promise.race + setTimeout



HTTP status codes

- Something we didn't mention in the previous lecture are HTTP status codes.
- HTTP response status codes indicate whether a specific HTTP request has been successfully completed. Responses are grouped in five classes:
 1. Informational responses (100 – 199)
 2. Successful responses (200 – 299)
 3. Redirection messages (300 – 399)
 4. Client error responses (400 – 499)
 5. Server error responses (500 – 599)
- There is no need to learn them by heart! You can always google them and find out what a certain code means!
- In order for this to work, it is the BACKEND responsibility to make this work. If the backend is written good, you will get correct status codes.



HTTP status codes

- Some of the most used ones are:
 - **200 OK** - The request succeeded. The result meaning of "success" depends on the HTTP method: GET,POST,PUT,PATCH,DELETE
 - **201 Created** - The request succeeded, and a new resource was created as a result. This is typically the response sent after POST requests, or some PUT requests.
 - **301 Moved Permanently** - The URL of the requested resource has been changed permanently. The new URL is given in the response.



HTTP status codes

6

- **400 Bad Request** The server cannot or will not process the request due to something that is perceived to be a client error (e.g., malformed request syntax, invalid request message framing, or deceptive request routing).
- **401 Unauthorized** Although the HTTP standard specifies "unauthorized", semantically this response means "unauthenticated". That is, the client must authenticate itself to get the requested response.
- **403 Forbidden** The client does not have access rights to the content; that is, it is unauthorized, so the server is refusing to give the requested resource. Unlike 401 Unauthorized, the client's identity is known to the server.
- **404 Not Found** The server cannot find the requested resource.
- **429 Too Many Requests** The user has sent too many requests in a given amount of time ("rate limiting")
- **500 Internal Server Error** The server has encountered a situation it does not know how to handle



HTTP status codes

- Why do we need them? , you might ask.
- We would usually use the different codes to show a different (error) message to the user or to take him to another flow.
- For example: if the creation of a new card was successful (201), we might show them a green successful message, or if the creation failed for whichever reason, we can show them a red error message and take him to a different flow. Ex: 401 Unauthorized, we will take the user to the sign-in page to authorize again!
- Where can we find the status codes? Run this code and find out!

```
fetch('https://jsonplaceholder.typicode.com/posts').then(res => console.log(res));
```



Example 9 (or homework)

1. Fetch all users from this api endpoint <https://jsonplaceholder.typicode.com/posts>
2. If the call was successful (check which status code), show a green toast for 2 seconds and close it.
3. Create a form for adding a new post.
4. If the creation of the new post was successful, show a green toast for 2 seconds and close it.
 - a. If the creation failed, show a red toast for 2 seconds and close it.

